```
--------------------------------------------------------------------------------
--Baseline.vhd
--------------------------------------------------------------------------------
-- One bit full adder
  entity FullAdder is
    port (
      A, B, Cin : in std_logic;
      S, Cout : out std_logic
      );
  end entity;

architecture structural of FullAdder is

  signal g, p : std_logic; -- Generate/Propagate signals

begin
  -- Generate and Propagate logic
  g <= A AND B;
  p <= A XOR B;

  -- Constructing the baseline outputs
  S <= p XOR Cin;
  Cout <= g OR (p AND Cin);

end architecture structural;
--------------------------------------------------------------------------------
--RippleAdder.vhd
--------------------------------------------------------------------------------

Entity Adder is
  Generic ( N : natural := 64 );
  Port ( A, B : in std_logic_vector( N-1 downto 0 );
   S : out std_logic_vector( N-1 downto 0 );
   Cin : in std_logic;
   Cout, Ovfl : out std_logic );
End Entity Adder;

architecture Baseline of Adder is

  component FullAdder is
    port (
      A, B, Cin : in std_logic;
      S, Cout   : out std_logic
    );
  end component;

  signal c : std_logic_vector(N downto 0); -- Internal carry chain array

begin

  c(0) <= Cin; -- Initial carry-in connected to start of the chain

  -- Loop to instatiate each adder
  GEN_ADDERS: for i in 0 to N-1 generate
     FA_i : FullAdder
       port map (
          A    => A(i),
          B    => B(i),
          Cin  => c(i),
          S    => S(i),
```

```
          Cout => c(i+1)
        );
  end generate GEN_ADDERS;

  Cout <= c(N);
  Ovfl <= c(N) xor c(N-1);

end architecture Baseline;

architecture FastRipple of Adder is

signal A_ext, B_ext, sum_ext : unsigned(N downto 0);

begin

-- Padding inputs A and B with '0' to account for overflow/Cout
  A_ext <= '0' & unsigned(A);
  B_ext <= '0' & unsigned(B);
  sum_ext <= A_ext + B_ext + ('0' & Cin); -- Sum now adding 65-bit unsigned numbers

  S <= std_logic_vector(sum_ext(N-1 downto 0));
  Cout <= std_logic(sum_ext(N)); -- Top bit is the Cout bit

  Ovfl <= (A(N-1) AND B(N-1) AND (NOT sum_ext(N-1))) OR ((NOT A(N-1)) AND (NOT B(N-
1)) AND sum_ext(N-1));

end architecture FastRipple;

--------------------------------------------------------------------------------
--CondSum.vhd
--------------------------------------------------------------------------------
-- Recursive N-bit Conditional-Sum Adder
architecture CondSum of Adder is

    constant M : natural := N/2;

    signal Sum_internal : std_logic_vector(N-1 downto 0);
    signal Sum0, Sum1 : std_logic_vector(M-1 downto 0);
    signal Cout0, Cout1 : std_logic;
    signal CarrySel : std_logic;
    signal CarryOut : std_logic;

begin


    -- Base case: 2-bit adder
    base_case: if N = 2 generate

        Sum_internal(0) <= A(0) xor B(0) xor Cin;

        Cout0 <= (A(0) and B(0)) or
                 (Cin and (A(0) xor B(0)));

        Sum_internal(1) <= A(1) xor B(1) xor Cout0;

        CarryOut <= (A(1) and B(1)) or
                    (Cout0 and (A(1) xor B(1)));

    end generate;

```

```
    -- Recursive case
    recursive_case: if N > 2 generate

        -- Lower half
        LowerAdder: entity work.Adder(CondSum)
            generic map (N => M)
            port map (
                A => A(M-1 downto 0),
                B => B(M-1 downto 0),
                Cin => Cin,
                S => Sum_internal(M-1 downto 0),
                Cout => CarrySel,
                Ovfl => open
            );

        -- Upper half assuming carry = 0
        UpperAdder_0: entity work.Adder(CondSum)
            generic map (N => M)
            port map (
                A => A(N-1 downto M),
                B => B(N-1 downto M),
                Cin => '0',
                S => Sum0,
                Cout => Cout0,
                Ovfl => open
            );

        -- Upper half assuming carry = 1
        UpperAdder_1: entity work.Adder(CondSum)
            generic map (N => M)
            port map (
                A => A(N-1 downto M),
                B => B(N-1 downto M),
                Cin => '1',
                S => Sum1,
                Cout => Cout1,
                Ovfl => open
            );

        -- Select correct upper-half result
        Sum_internal(N-1 downto M) <=
            Sum0 when CarrySel = '0' else Sum1;

        CarryOut <= Cout0 when CarrySel = '0' else Cout1;

    end generate;

    -- Final outputs
    S <= Sum_internal;
    Cout <= CarryOut;

    -- Signed overflow
    Ovfl <= (A(N-1) and B(N-1) and not Sum_internal(N-1)) or
            (not A(N-1) and not B(N-1) and Sum_internal(N-1));

end architecture CondSum;


--------------------------------------------------------------------------------
--CarryLookahead.vhd
```

```vhdl
----------------------------------------------------------------------------------
-- Design Candidate - Multi-level look ahead (Carry-lookahead) Adder Architecture

-- Carry look ahead unit (LCU)
entity LCU is
    port (
        G      : in  std_logic_vector(3 downto 0);
        P      : in  std_logic_vector(3 downto 0);
        Cin    : in  std_logic;
        C      : out std_logic_vector(3 downto 1);
        Cout   : out std_logic;
        PGroup : out std_logic;
        GGroup : out std_logic
    );
end entity LCU;

-- Architecture to generate 4-bit LCU at bitwise level
architecture structural of LCU is
begin
    C(1) <= G(0) or (P(0) and Cin);
    C(2) <= G(1) or (P(1) and G(0)) or (P(1) and P(0) and Cin);
    C(3) <= G(2) or (P(2) and G(1)) or (P(2) and P(1) and G(0)) or (P(2) and P(1) and P(0) and Cin);
    Cout <= G(3) or (P(3) and G(2)) or (P(3) and P(2) and G(1)) or (P(3) and P(2) and P(1) and G(0)) or (P(3) and P(2) and P(1) and P(0) and Cin);

    PGroup <= P(3) and P(2) and P(1) and P(0);
    GGroup <= G(3) or (P(3) and G(2)) or (P(3) and P(2) and G(1)) or (P(3) and P(2) and P(1) and G(0));
end architecture structural;

-- Carry-lookahead architecture to instatiate the 3 levels needed
architecture CarryLookahead of Adder is

    component LCU is
        port (
            G      : in  std_logic_vector(3 downto 0);
            P      : in  std_logic_vector(3 downto 0);
            Cin    : in  std_logic;
            C      : out std_logic_vector(3 downto 1);
            Cout   : out std_logic;
            PGroup : out std_logic;
            GGroup : out std_logic
        );
    end component;

    signal g_int, p_int : std_logic_vector(3 downto 0);
    signal c_int        : std_logic_vector(3 downto 1);
    signal carry_chain  : std_logic_vector(4 downto 0);
    signal cout_int     : std_logic;

    -- Internal signal for overflow with the sum vector
    signal s_int        : std_logic_vector(N-1 downto 0);

    signal unused_pg_sig, unused_gg_sig : std_logic;

begin

    assert N = 4 report "CarryLookahead architecture configured for N=4 blocks." severity note;
```

```
236
237    -- Generating the group signals to go down into the 4 4-bit LCUs
238     carry_chain(0)          <= Cin;
239     carry_chain(3 downto 1) <= c_int;
240     carry_chain(4)          <= cout_int;
241     Cout                    <= cout_int;
242
243     gen_gp: for i in 3 downto 0 generate
244         g_int(i) <= A(i) and B(i);
245         p_int(i) <= A(i) xor B(i);
246     end generate gen_gp;
247
248    -- Instatiating the 4 4-bit LCUs to group the pairs of G/P signals
249     CLU_INST : LCU
250         port map (
251             G      => g_int,
252             P      => p_int,
253             Cin    => Cin,
254             C      => c_int,
255             Cout   => cout_int,
256             PGroup => unused_pg_sig,
257             GGroup => unused_gg_sig
258         );
259
260     gen_sum: for i in 3 downto 0 generate
261         s_int(i) <= p_int(i) xor carry_chain(i);
262     end generate gen_sum;
263
264     gen_zero_upper: if N > 4 generate
265         zero_s: for i in N-1 downto 4 generate
266             s_int(i) <= '0';
267         end generate;
268     end generate;
269
270     -- Drive output port S from internal signal
271     S <= s_int;
272
273     -- Evaluate overflow using internal s_int instead of the output port S
274     Ovfl <= (A(N-1) and B(N-1) and not s_int(N-1)) or (not A(N-1) and not B(N-1) and
s_int(N-1));
275
276 end architecture CarryLookahead;
277
278
279 ----------------------------------------------------------------------------------
280 --CarrySelect.vhd
281 ----------------------------------------------------------------------------------
282
283 architecture CarrySelect of Adder is
284
285     -- Four 16-bit blocks for a 64-bit adder
286     constant BLOCK_SIZE : natural := 16;
287     constant NUM_BLOCKS : natural := N / BLOCK_SIZE;
288
289     -- Internal signals for computed sums and carries for both Cin=0 and Cin=1 paths
290     signal s_path0, s_path1       : std_logic_vector(N-1 downto 0);
291     signal cout_path0, cout_path1 : std_logic_vector(NUM_BLOCKS-1 downto 0);
292
293     -- Actual carry chain vector passing between blocks
294     signal block_carry            : std_logic_vector(NUM_BLOCKS downto 0);
```

```
295
296     -- Internal signal for final sum and overflow evaluation
297     signal s_int                  : std_logic_vector(N-1 downto 0);
298
299 begin
300
301     -- Connect initial external carry-in (C0)
302     block_carry(0) <= Cin;
303
304     -- Generate blocks
305     gen_csla_blocks: for i in 0 to NUM_BLOCKS - 1 generate
306
307         -- Index ranges for the current block
308         constant LOW_IDX  : natural := i * BLOCK_SIZE;
309         constant HIGH_IDX : natural := (i + 1) * BLOCK_SIZE - 1;
310
311         signal a_block, b_block : std_logic_vector(BLOCK_SIZE-1 downto 0);
312         signal sum0_block, sum1_block : std_logic_vector(BLOCK_SIZE-1 downto 0);
313         signal c0_block, c1_block     : std_logic;
314
315     begin
316         a_block <= A(HIGH_IDX downto LOW_IDX);
317         b_block <= B(HIGH_IDX downto LOW_IDX);
318
319         -- Block assuming Cin = '0'
320         process(a_block, b_block)
321             variable sum_v : unsigned(BLOCK_SIZE downto 0);
322         begin
323             sum_v := ('0' & unsigned(a_block)) + ('0' & unsigned(b_block));
324             sum0_block <= std_logic_vector(sum_v(BLOCK_SIZE-1 downto 0));
325             c0_block   <= std_logic(sum_v(BLOCK_SIZE));
326         end process;
327
328       -- Block assuming Cin = '1'
329         process(a_block, b_block)
330             variable sum_v : unsigned(BLOCK_SIZE downto 0);
331         begin
332             sum_v := ('0' & unsigned(a_block)) + ('0' & unsigned(b_block)) + 1;
333             sum1_block <= std_logic_vector(sum_v(BLOCK_SIZE-1 downto 0));
334             c1_block   <= std_logic(sum_v(BLOCK_SIZE));
335         end process;
336
337         -- Store block results into global vectors
338         s_path0(HIGH_IDX downto LOW_IDX)    <= sum0_block;
339         s_path1(HIGH_IDX downto LOW_IDX)    <= sum1_block;
340         cout_path0(i)                       <= c0_block;
341         cout_path1(i)                       <= c1_block;
342
343       -- MUX to select correct Cout for next block based on current Cin
344         block_carry(i+1) <= c0_block when block_carry(i) = '0' else c1_block;
345
346       -- MUX to select correct sum bits
347         process(block_carry(i), sum0_block, sum1_block)
348         begin
349             if block_carry(i) = '0' then
350                 s_int(HIGH_IDX downto LOW_IDX) <= sum0_block;
351             else
352                 s_int(HIGH_IDX downto LOW_IDX) <= sum1_block;
353             end if;
354         end process;
```

```
355
356     end generate gen_csla_blocks;
357
358     -- Final Outputs
359     S    <= s_int;
360     Cout <= block_carry(NUM_BLOCKS);
361
362     -- Signed Overflow detection
363     Ovfl <= (A(N-1) and B(N-1) and not s_int(N-1)) or (not A(N-1) and not B(N-1) and
    s_int(N-1));
364
365 end architecture CarrySelect;
366
367 ----------------------------------------------------------------------------------
368 --CarrySkip.vhd
369 ----------------------------------------------------------------------------------
370
371 architecture CarrySkip of Adder is
372
373     constant BLOCK_SIZE : positive := 4;
374     constant NUM_BLOCKS : natural := (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
375
376     signal block_carry : std_logic_vector(NUM_BLOCKS downto 0);
377     signal s_int       : std_logic_vector(N-1 downto 0);
378
379 begin
380
381     assert N > 0
382         report "CarrySkip requires N >= 1."
383         severity failure;
384
385     block_carry(0) <= Cin;
386
387     gen_blocks: for i in 0 to NUM_BLOCKS - 1 generate
388         signal ripple_carry    : std_logic_vector(BLOCK_SIZE downto 0);
389         signal propagate_chain : std_logic_vector(BLOCK_SIZE downto 0);
390     begin
391         ripple_carry(0)    <= block_carry(i);
392         propagate_chain(0) <= '1';
393
394         gen_bits: for j in 0 to BLOCK_SIZE - 1 generate
395             constant BIT_INDEX : natural := i * BLOCK_SIZE + j;
396         begin
397             active_bit: if BIT_INDEX < N generate
398                 signal p, g : std_logic;
399             begin
400                 p <= A(BIT_INDEX) xor B(BIT_INDEX);
401                 g <= A(BIT_INDEX) and B(BIT_INDEX);
402
403                 s_int(BIT_INDEX) <= p xor ripple_carry(j);
404                 ripple_carry(j+1) <= g or (p and ripple_carry(j));
405                 propagate_chain(j+1) <= propagate_chain(j) and p;
406             end generate active_bit;
407
408             -- Pass through unused positions in a short final block.
409             padding_bit: if BIT_INDEX >= N generate
410                 ripple_carry(j+1) <= ripple_carry(j);
411                 propagate_chain(j+1) <= propagate_chain(j);
412             end generate padding_bit;
413         end generate gen_bits;
```

```
414
415         -- When every active bit propagates, bypass the block's ripple carry.
416         block_carry(i+1) <= block_carry(i)
417             when propagate_chain(BLOCK_SIZE) = '1'
418             else ripple_carry(BLOCK_SIZE);
419     end generate gen_blocks;
420
421     S    <= s_int;
422     Cout <= block_carry(NUM_BLOCKS);
423
424     -- Signed overflow: equal operand signs and a different result sign.
425     Ovfl <= (A(N-1) and B(N-1) and not s_int(N-1)) or
426             (not A(N-1) and not B(N-1) and s_int(N-1));
427
428 end architecture CarrySkip;
429
```